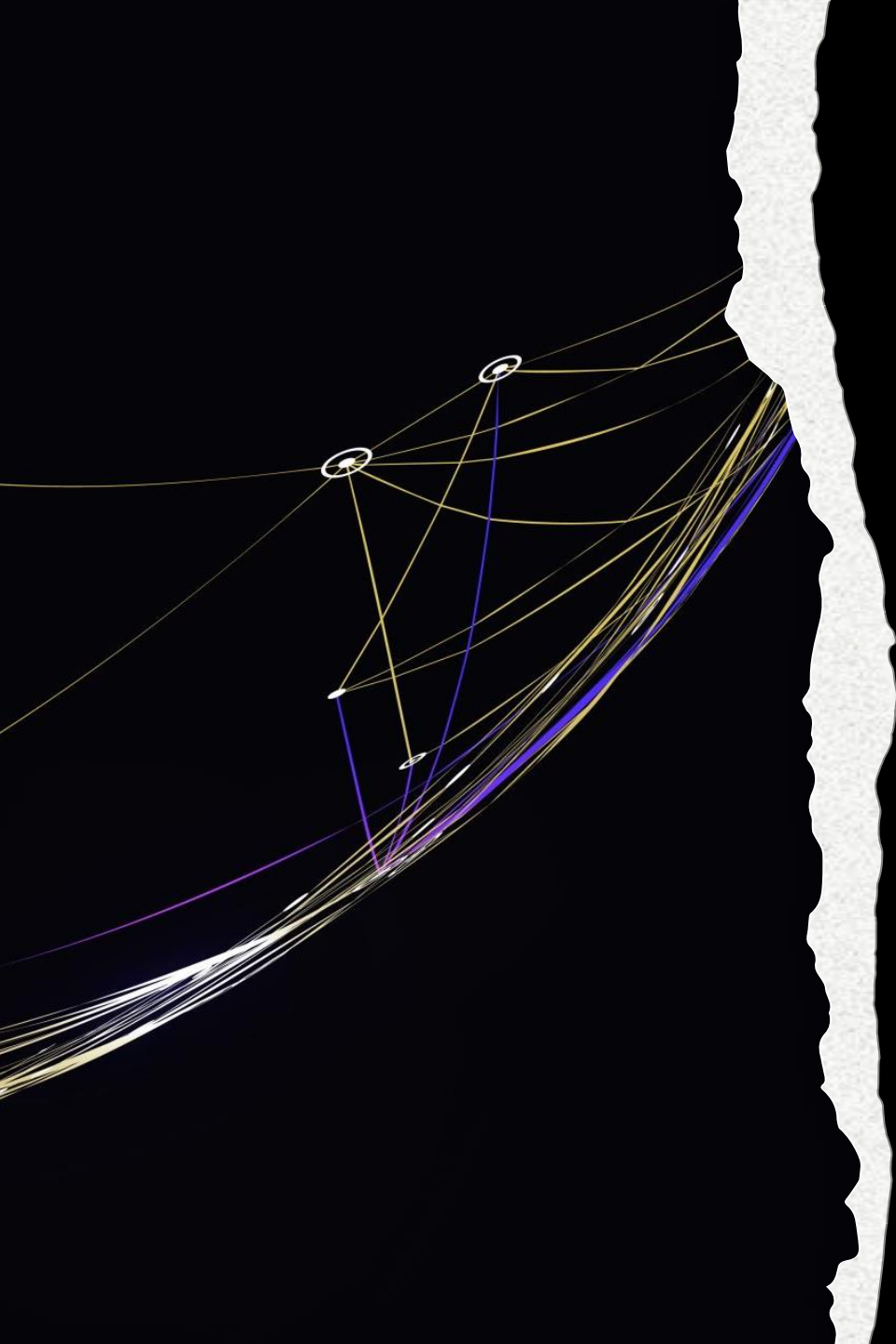




Algorytmy geometryczne 2024/2025 projekt

Temat: Otoczka wypukła dla zbioru
punktów w przestrzeni
dwuwymiarowej

AUTORZY: MATEUSZ WÓJCIK I BŁAŻEJ NAZIEMIEC

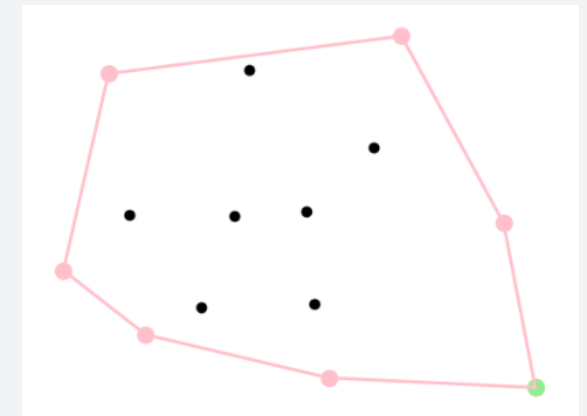
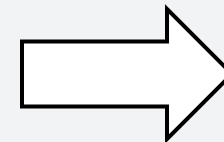
GRUPA 1

Przedstawienie problemu:

Dany mamy zbiór punktów S rozmieszczony w układzie kartezjańskim $x y$. Naszym problemem jest znalezienie najmniejszego wielokąta wypukłego obejmującego wszystkie punkty danego zbioru. Nazywamy to otoczką wypukłą.

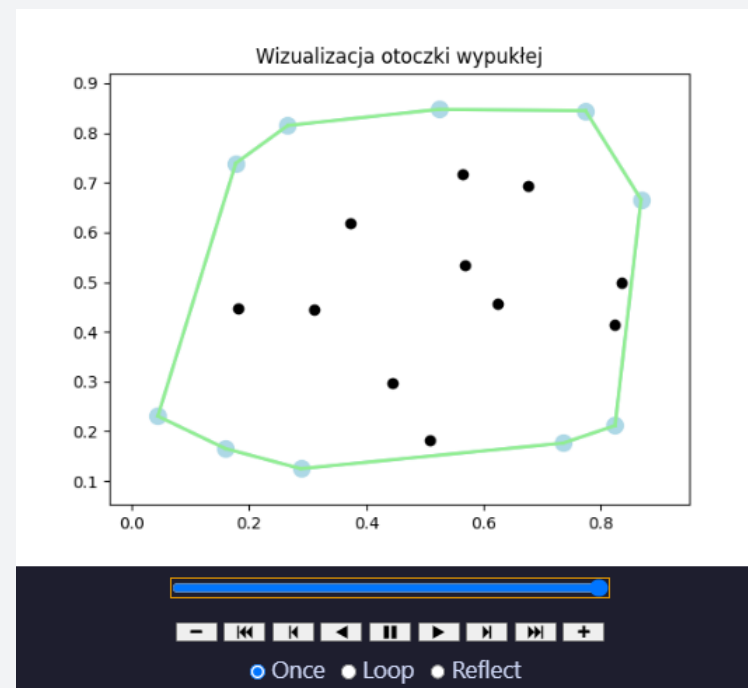
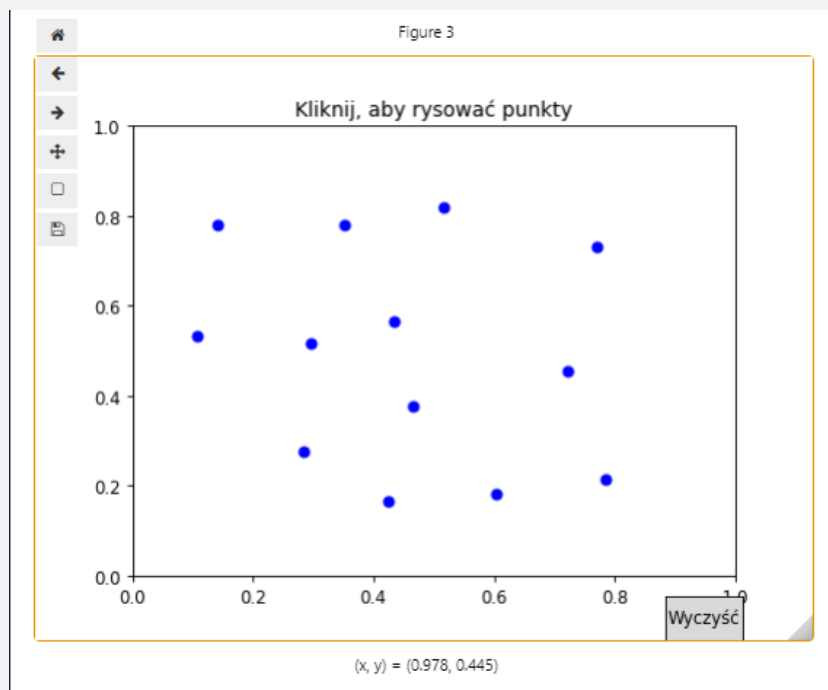
Aby uzyskać taki zbiór wykorzystywane są algorytmy:

- Przyrostowy
- Górna i dolna otoczka
- Quickhull
- Dziel i rządź
- Chan'a
- Grahama
- Jarvisa



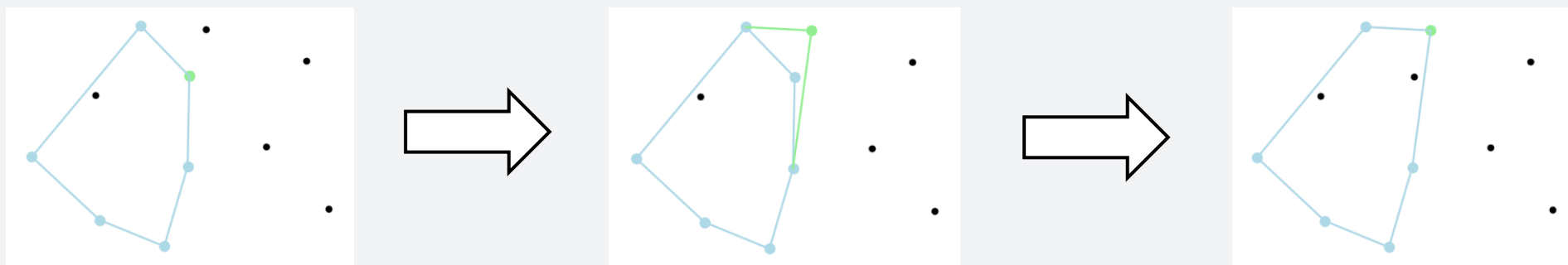
Użyte struktury danych

- Klasa Point reprezentuje punkt w przestrzeni 2D, przechowujący współrzędne x i y.
- Klasa Segment reprezentuje odcinek w przestrzeni 2D, łączący dwa punkty
- Klasa PolygonDrawer umożliwia interaktywne rysowanie i zapisywanie współrzędnych punktów w przestrzeni 2D
- Klasa ConvexHullVisualizer służy do wizualizacji algorytmów wyznaczania otoczki wypukłej.



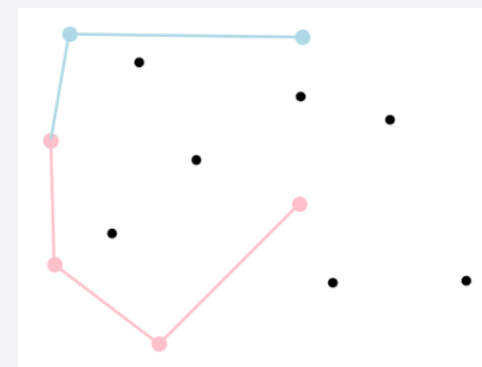
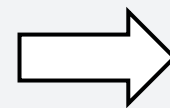
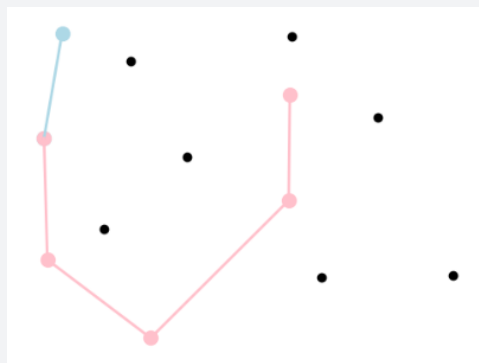
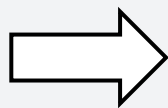
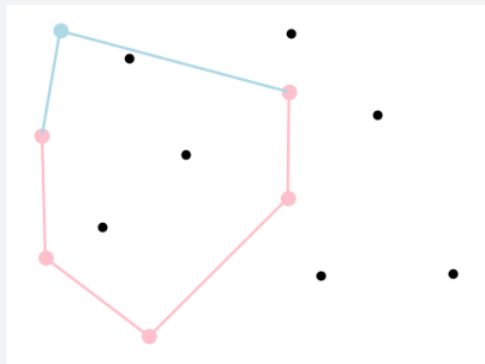
Algorytm przyrostowy

Algorytm przyrostowy polega na początkowym posortowaniu wszystkich punktów zbioru S rosnąco względem współrzędnej x oraz y punktów, a następnie wyznaczeniu otoczki składającej się z dwóch pierwszych punktów posortowanego zbioru. W następnych krokach dla każdego kolejnego punktu szukamy wśród wcześniej wyznaczonych punktów takie, z którymi punkt ten tworzyłby tymczasową otoczkę dla aktualnie wszystkich sprawdzonych punktów.. Wykorzystywana jest do tego funkcja **orient**, która za pomocą wyznacznika macierzy 3×3 jest w stanie określić, czy 3 punkty tworzą lewo\prawostronny skręt lub czy 3 punkty nie są współliniowe. Złożoność tego algorytmu to $O(n \log n)$, a w najgorszym przypadku $O(n^2)$.



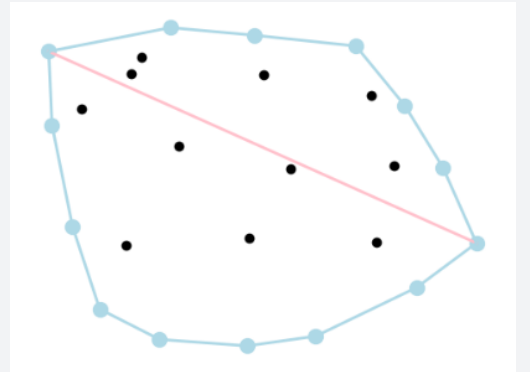
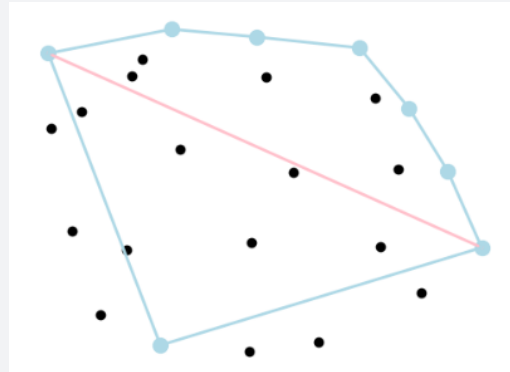
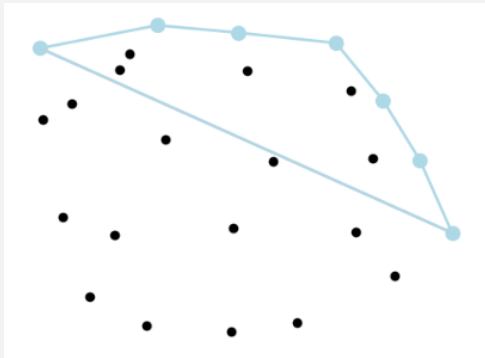
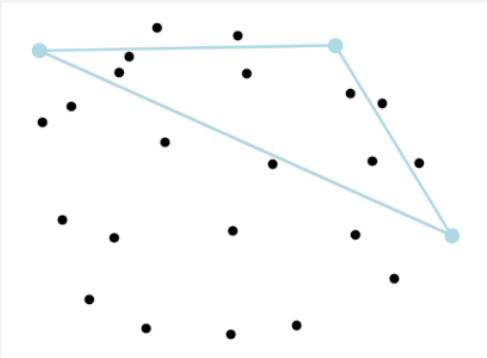
Algorytm górna i dolna otoczka

Jak sama nazwa wskazuje – algorytm działa poprzez podzielenie otoczki na górną oraz dolną część. Na początku należy posortować zbiór punktów rosnąco względem współrzędnej x i y punktów. W następnym kroku tworzymy górną i dolną otoczkę, do której należą dwa pierwsze punkty z posortowanej listy punktów. Następnie dla każdego punktu sprawdzamy, szukamy ostatniego punktu w górnej/dolnej otoczce, z którym nie tworzy lewostronnego skrętu. Na sam koniec łączymy obie otoczki i w efekcie otrzymujemy finalną otoczkę. Złożoność tego algorytmu to $O(n \log n)$.



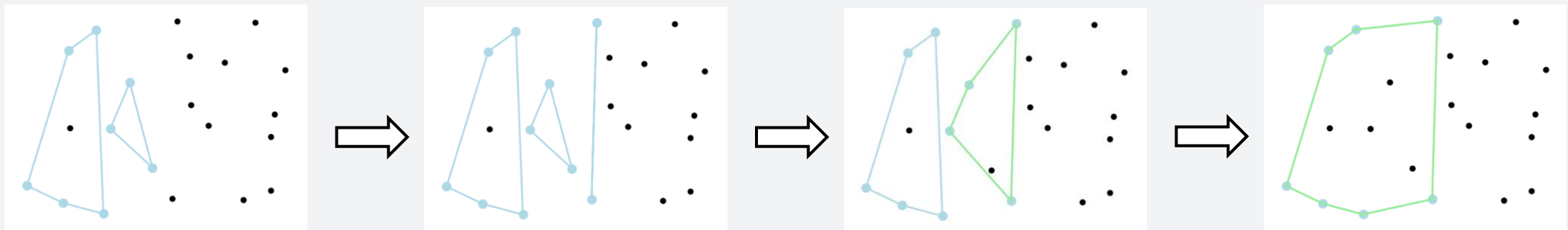
Algorytm Quickhull

Algorytm QuickHull rozpoczyna się od znalezienia najbardziej skrajnych punktów w lewo i w prawo, które tworzą początkowy odcinek otoczki wypukłej. Następnie zbiór punktów dzielimy na te leżące powyżej i poniżej tego odcinka. Rekurencyjnie dla każdego z tych zbiorów wybieramy punkt, który znajduje się najdalej od prostej wyznaczonej przez odcinek – punkt ten jest dodawany do otoczki. Powstają nowe odcinki łączące znaleziony punkt z punktami pierwotnego odcinka. Procedurę powtarzamy, wybierając punkty leżące powyżej nowych odcinków, aż do wyczerpania zbiorów punktów. Analogicznie przetwarzamy punkty poniżej odcinków, "odwracając" procedurę. W ten sposób algorytm buduje otoczkę, rekurencyjnie dzieląc i analizując punkty w mniejszych podzbiorach.



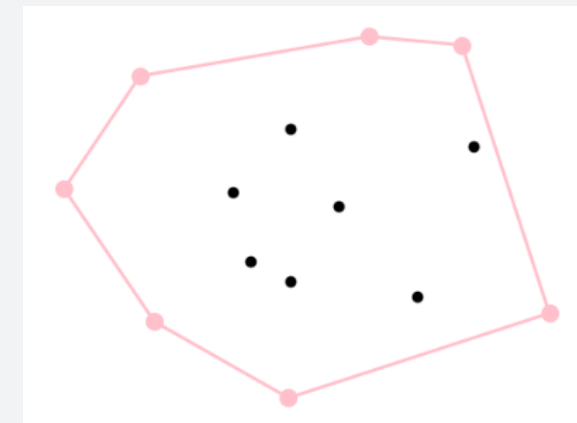
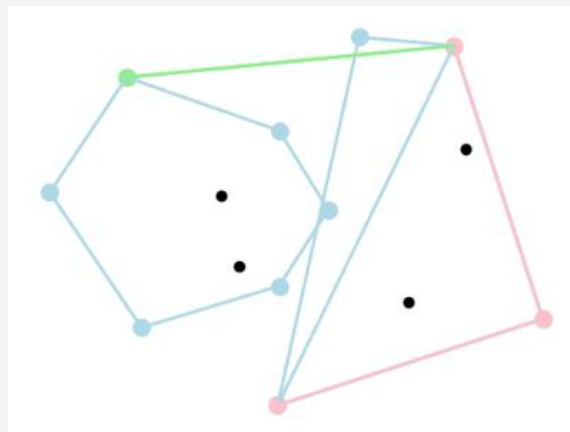
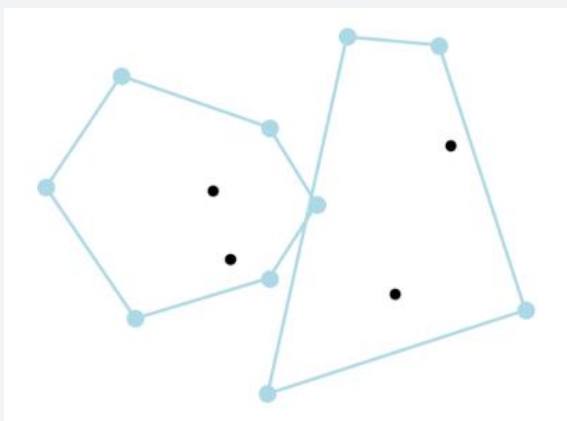
Algorytm dziel i rządź

Algorytm ten wykorzystuje tę samą taktykę co algorytm sortowania MergeSort – „divide-and-conquer”. Na początku algorytm sortuje punkty rosnąco po współrzędnych x i y . Następnie algorytm rekurencyjnie dzieli punkty na maksymalnie 3 elementowe podgrupy, w których wyznacza otoczkę. Kolejnym krokiem jest łączenie dwóch otoczek. W tym celu algorytm szuka dwóch górnych punktów, które tworzą górną styczną do obu otoczek i analogicznie dwóch dolnych punktów dla dolnej stycznej. Aby to zrealizować szukamy pierwszego punktu, który tworzy prawostronny skręt z wyznaczonym punktem z drugiej otoczki oraz swoim górnym/dolnym sąsiadem. Po wyznaczeniu punktów nowych stycznych łączymy obie otoczki. Złożoność tego algorytmu to $O(n \log n)$



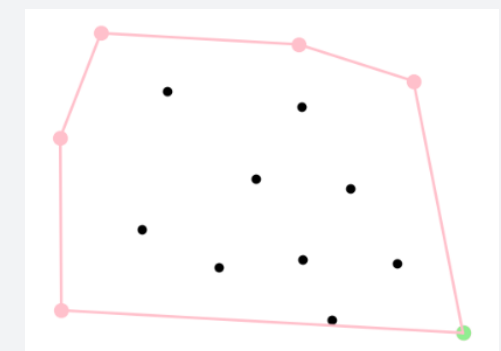
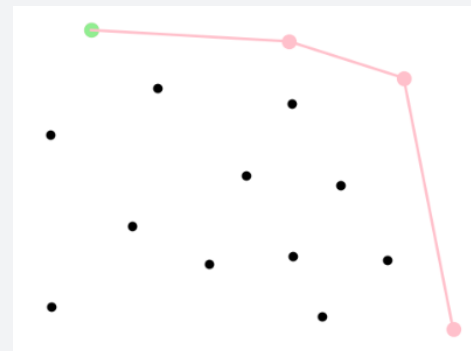
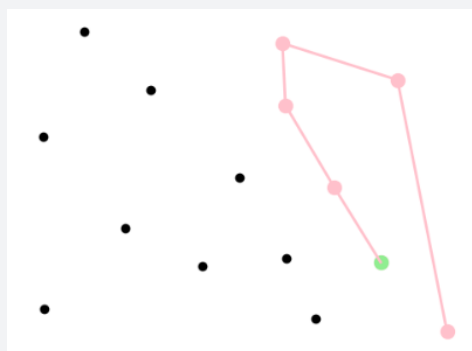
Algorytm Chan'a

Najważniejszą rzeczą, aby algorytm Chan'a działał w odpowiedniej złożoności czasowej jest m – aproksymacja liczby punktów w końcowej otoczce. Na początku dzielimy wszystkie punkty na $r = \lceil n/m \rceil$ grup liczących maksymalnie m punktów. W następnym kroku dla każdej grupy wyznaczamy otoczkę wykorzystując do tego algorytm Grahama. Dzięki tej operacji każda podgrupa posiada posortowane punkty oraz część punktów została wyeliminowana z możliwości zostania wierzchołkiem otoczki. Następnie wyznaczamy punkt, który ma najmniejszą współrzędną y (w przypadku kilku takich punktów wybierany jest ten z najmniejszą współrzędną x) oraz m razy wyznaczamy kolejne punkty otoczki. Dzięki faktowi, że w każdej podotoczce punkty są posortowane możemy wyszukiwaniem binarnym znaleźć punkt, który tworzy najmniejszy kąt z aktualnie dwoma ostatnimi punktami w otoczce końcowej. Działanie to jest analogiczne do działania algorytmu Jarvisa. Złożoność takiego algorytmu to $O(n \log m)$.



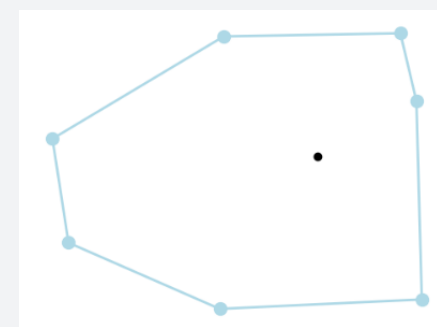
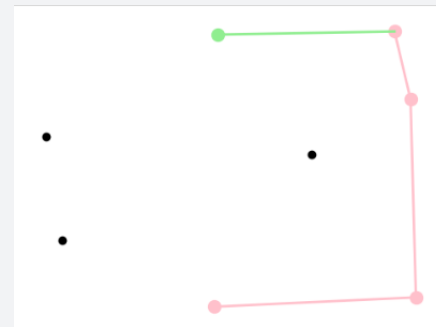
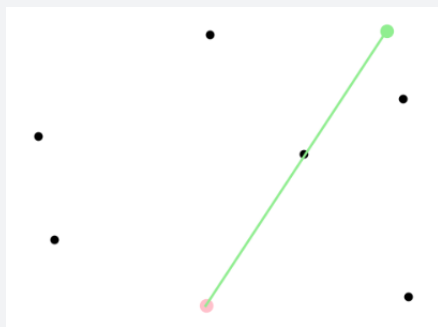
Algorytm Grahama

Na samym początku algorytm znajduje punkt o najmniejszej współrzędnej y (nazwijmy go p_1)(w przypadku kilku punktów wybiera ten z najmniejszą współrzędną x). W następnym kroku wyznacza on dla każdego punktu kąt, jaki tworzy prosta zawierająca ten punkt oraz p_1 z osią x oraz odległość między tymi punktami. W kolejnym kroku algorytm sortuje te punkty właśnie po kącie oraz odległości, aby później usunąć wszystkie punkty, które tworzą ten sam kąt z punktem p_1 zostawiając tylko ten, który jest najbardziej od niego oddalony. Ostatecznie algorytm sprawdza, czy 3 ostatnie punkty tworzą skręt prawostronny. Jeśli nie, to ostatni punkt w otoczce jest usuwany i sprawdzamy usuwamy kolejne punkty, dopóki nie osiągniemy skrętu prawostronnego. Po sprawdzeniu wszystkich punktów algorytm zwraca finalną otoczkę. Złożoność tego algorytmu wynosi $O(n \log n)$.



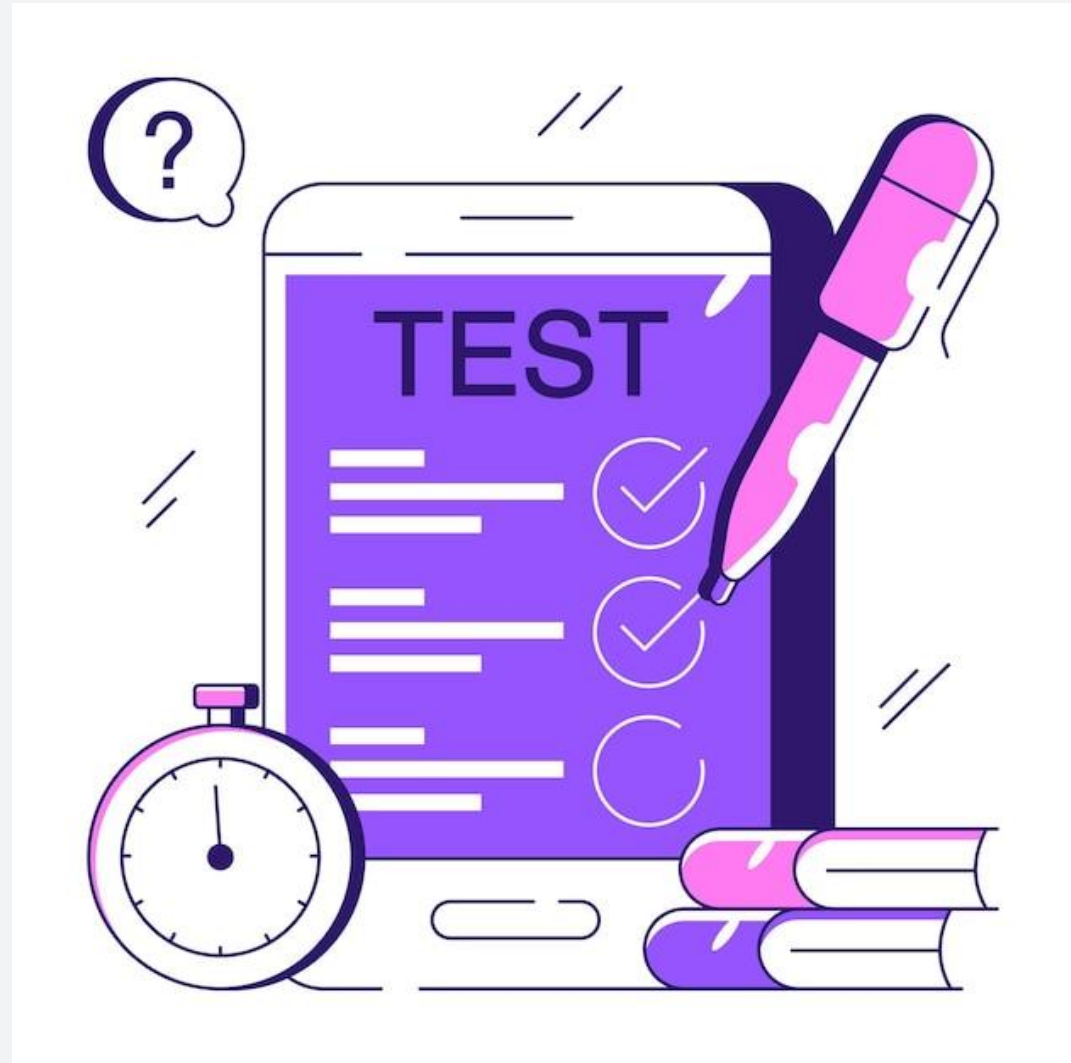
Algorytm Jarvisa

Algorytm ten inaczej nazywany jest algorytmem owijania prezentu. Na samym początku znajdujemy punkt o najmniejszej współrzędnej y (w przypadku kilku punktów wybiera ten z najmniejszą współrzędną x). W kolejnym ruchu algorytm sprawdza każdy inny punkt i sprawdza, jaki typu skrętu on tworzy. Jeśli skręt jest prawostronny lub punkty są współliniowe, ale odległość aktualnie rozpatrywanego punktu i ostatniego w otoczce jest większa od odległości ostatnio wybranego za możliwego do tworzenia otoczki oraz ostatniego w otoczce, to ten punkt jest uznawany za możliwy do utworzenia otoczki. Proces ten powtarzamy dla każdego nowego wierzchołka otoczki. Złożoność tego algorytmu to $O(nh)$, gdzie h to liczba punktów w finalnej otoczce. Dla przypadku, gdy każdy punkt należy do finalnej otoczki złożoność ta rośnie to $O(n^2)$.

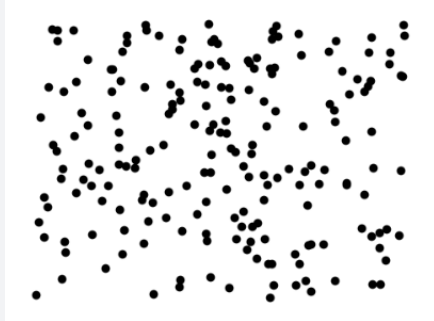


Przygotowane zbiory testowe

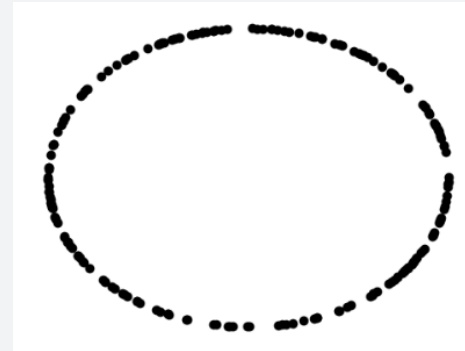
W ramach testowania poprawności algorytmu przygotowano 4 zbiorów sprawdzających poprawność algorytmu oraz 4 typy zbiorów sprawdzających czas działania poszczególnych algorytmów ze zmienną liczbą punktów w zbiorze



Testy sprawdzające poprawność



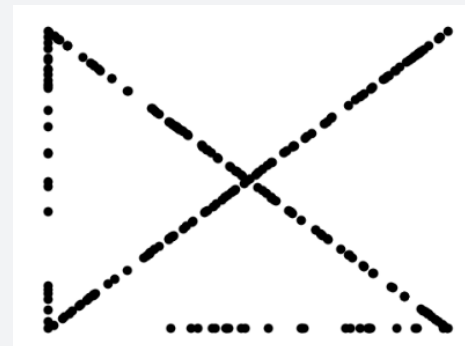
Test 1: 200 losowo
rozmieszczonych punktów



Test 2: 200 punktów
rozmieszczonych na okręgu



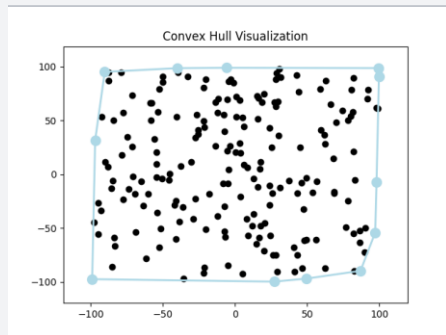
Test 3: 200 punktów
rozmieszczonych na
bokach prostokąta



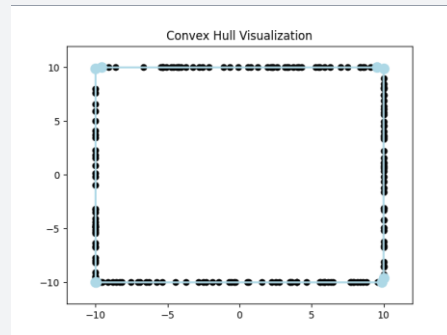
Test 4: 200 punktów
rozmieszczonych na
przekątnych kwadratu, 50
punktów na jego bokach
oraz w każdym rogu
kwadratu

Testy sprawdzające czas działania algorytmów

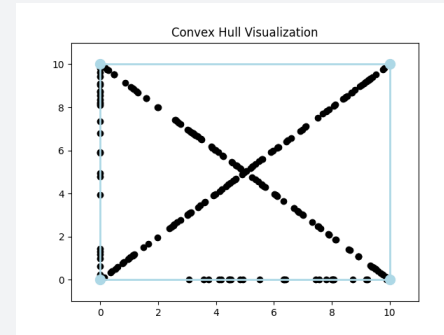
- Zbiór A: losowo rozmieszczone 100, 500, 1000, 5000, 10000 i 50000 punktów
- Zbiór B: 100, 500, 1000, 5000, 10000 i 50000 punktów rozmieszczonych na bokach prostokąta
- Zbiór C: 100, 500, 1000, 5000, 10000 i 50000 punktów rozmieszczonych na przekątnych oraz dwóch bokach kwadratu
- Zbiór D: 100, 500, 1000, 5000 punktów rozmieszczonych na okręgu



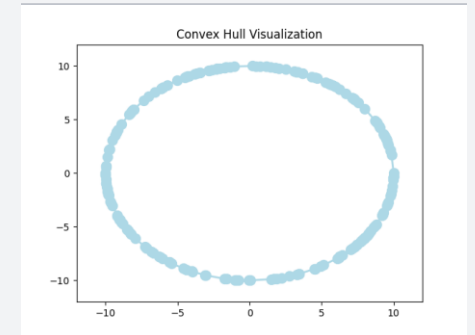
Zbiór A



Zbiór B



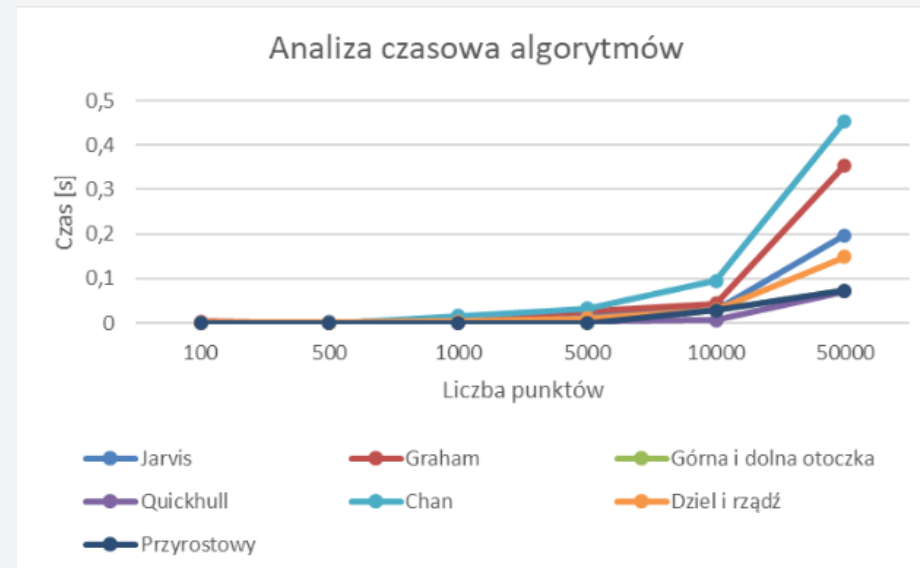
Zbiór C



Zbiór D

Wyniki czasu działania algorytmów dla zestawu 1

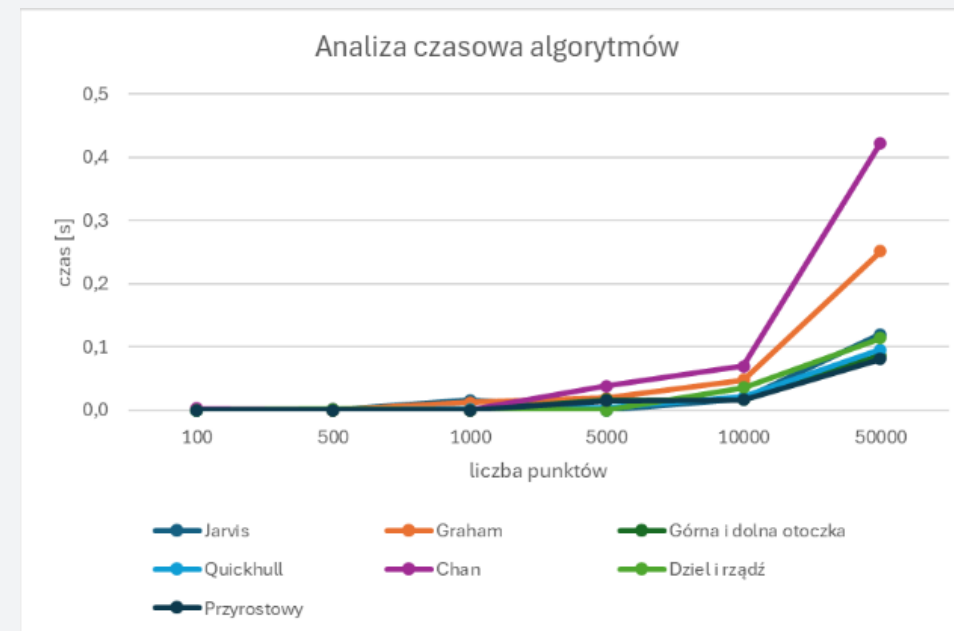
Najwolniejszy okazał się algorytm Chan'a, ponieważ jego implementacja w tym przypadku nie jest optymalna. Chociaż jego złożoność powinna wskazywać, iż będzie to najszybszy algorytm, z powodu losowej liczby punktów tworzących otoczkę w testach to on okazał się być najwolniejszy. Z kolei najszybszy okazał się algorytm Quickhull, który skutecznie radzi sobie w sytuacjach, gdzie liczba punktów na otoczce wypukłej jest stosunkowo mała. W takich przypadkach heurystyki Quickhull minimalizują liczbę wykonywanych operacji. Podobny czas osiągnął algorytm przyrostowy. Jest to spowodowane stosunkowo niewielką liczbą punktów w finalnej otoczce. Reszta algorytmów wykazywała zbliżone czasy działania, co wskazuje na ich porównywalną efektywność w tym scenariuszu. Test ten pozwalał wykazać różnice głównie w sposobie implementacji, ze względu na zbiór znacząco nie faworyzujący żadnych algorytmów.



Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0035	0.0	0.0	0.0	0.001	0.0
500	0.0017	0.0	0.0	0.001	0.0	0.001	0.0
1000	0.0033	0.0078	0.0	0.001	0.0164	0.003	0.0
5000	0.0202	0.0273	0.0	0.005	0.0331	0.0112	0.0
10000	0.029	0.0442	0.028	0.0063	0.0947	0.0296	0.028
50000	0.197	0.3538	0.0721	0.0715	0.4535	0.1489	0.0721

Wyniki czasu działania algorytmów dla zestawu 2

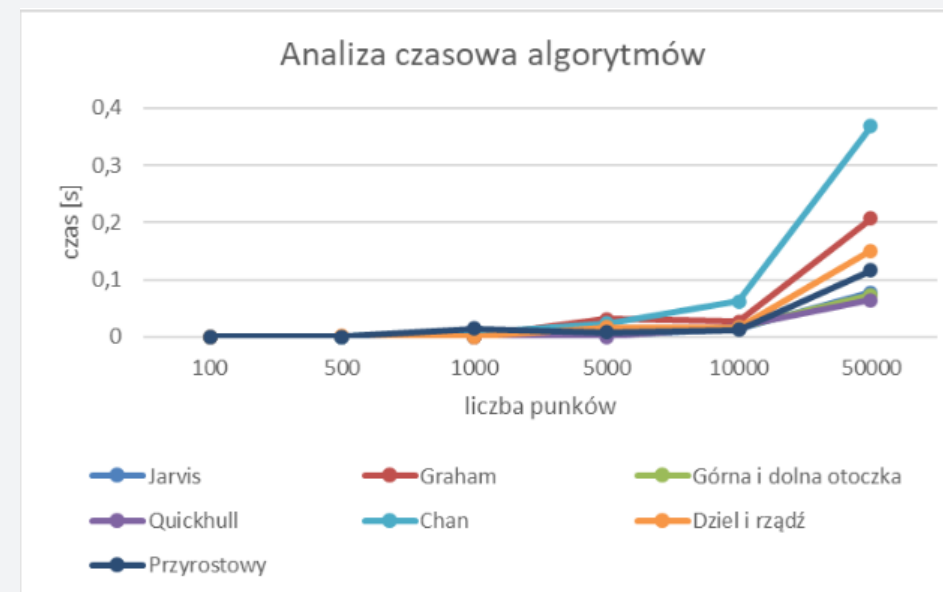
Przedstawione na rysunku oraz w tabeli dane pokazują, że najgorszy czas osiągnął algorytm Chan'a. Jest to spowodowane dużą liczbą otoczek, gdzie dla każdej z nich algorytm musi wyznaczyć punkt tworzący jak najmniejszy kąt z istniejącą już otoczką. Mimo, iż złożoność tego algorytmu jest najmniejsza ze wszystkich, to w praktyce nie zawsze jest to najszybszy sposób rozwiązania problemu. Najszybsze okazały się algorytmy górnej i dolnej otoczki oraz przyrostowy. Dzięki stosunkowo niewielkiej liczbie punktów tworzących końcową otoczkę, algorytmy te były w stanie osiągnąć najlepsze rezultaty czasowe i zdecydowanie były najlepszym rozwiązaniem dla tego zbioru testów.



Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0	0.0	0.0	0.0026	0.0	0.0
500	0.0	0.0	0.0	0.001	0.0005	0.0017	0.0
1000	0.0159	0.0122	0.0	0.002	0.0	0.0	0.0
5000	0.0	0.0191	0.0163	0.004	0.0381	0.0	0.0144
10000	0.0184	0.0482	0.0167	0.0213	0.0692	0.0359	0.0163
50000	0.1205	0.2511	0.0872	0.0954	0.4228	0.1136	0.0813

Wyniki czasu działania algorytmów dla zestawu 3

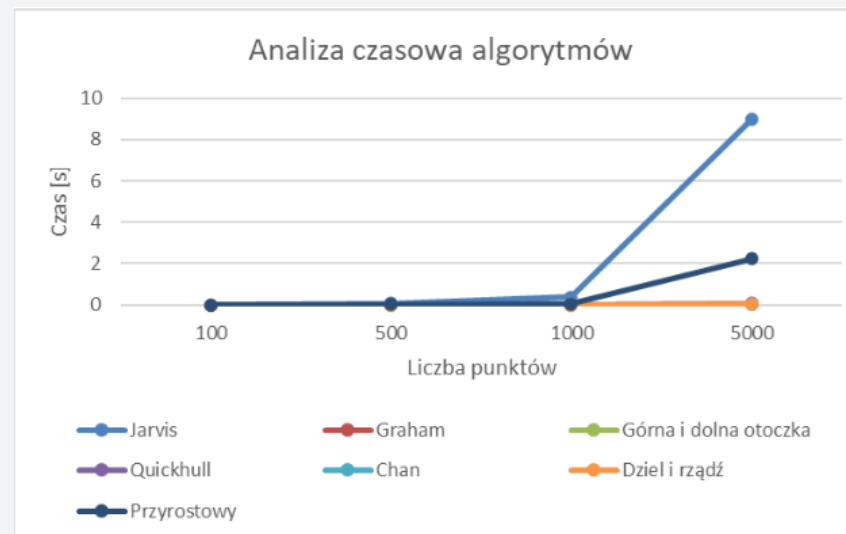
W tym przypadku najgorzej wypadł algorytm Chan'a. Jego słaba wydajność wynika z dużej liczby podotoczek, gdzie dla każdej musi on odnaleźć najbardziej ekstremalny punkt co bardzo spowalnia jego działanie. Quickhull ponownie był najszybszy, ponieważ efektywnie przetwarza przypadki, gdzie liczba punktów na otoczce jest mała. Reszta algorytmów wykazywała zbliżone czasy działania, co oznacza, że radzą sobie w podobny sposób w tym układzie punktów.



Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0	0.0	0.0	0.0	0.0	0.0	0.0
500	0.0	0.0	0.0	0.0	0.0	0.0011	0.0
1000	0.0022	0.0	0.0	0.0	0.0079	0.0	0.015
5000	0.006	0.0315	0.0067	0.0	0.0237	0.0166	0.008
10000	0.0147	0.0271	0.0161	0.0183	0.0625	0.017	0.0128
50000	0.078	0.2072	0.073	0.0647	0.3681	0.1511	0.1169

Wyniki czasu działania algorytmów dla zestawu 4

Najgorzej wypadł algorytm Jarvisa, którego złożoność $O(nh)$ w tej sytuacji przekształca się w $O(n^2)$, ponieważ $h = n$. W przypadku, gdy wszystkie punkty należą do otoczki, każdy krok wymaga przetworzenia wszystkich punktów, co znacząco wydłuża czas działania. Najlepiej poradził sobie algorytm Chan'a. Jego struktura, podział na mniejsze podgrupy, wyznaczeniu otoczki algorytmem Grahama a następnie łączenie w tym przypadku skraca się tylko do uruchomienia algorytmu Grahama. Z racji, iż wiemy, że każdy punkt należy do finalnej otoczki możemy podać dokładne m , a zatem uzyskać najlepszą złożoność. Algorytm przyrostowy zajął relatywnie dużo czasu. Wynika to z jego iteracyjnego charakteru: dodaje on kolejne punkty i za każdym razem aktualizuje otoczkę. W przypadku, gdy wszystkie punkty należą do otoczki, staje się to bardzo czasochłonnym procesem. Ten test kompleksowo weryfikował zdolność algorytmów do efektywnego wyznaczania otoczki wypukłej, gdy dominująca liczba punktów należy do tej otoczki.



Liczba punktów	Jarvis	Graham	Górna i dolna otoczka	Quickhull	Chan	Dziel i rządź	Przyrostowy
100	0.0012	0.0	0.0	0.0	0.0	0.0	0.0
500	0.0837	0.0	0.0	0.0167	0.0	0.013	0.0121
1000	0.3791	0.0	0.0	0.015	0.0156	0.0019	0.0213
5000	8.9774	0.0337	0.0168	0.0687	0.0156	0.0293	0.7661

Podsumowanie wyników testów

Wyniki testów pokazują, że niezwykle istotną rzeczą jest odpowiedni dobór algorytmów do struktury danych wejściowych. Algorytm Quickhull wykazał największą uniwersalność, radząc sobie najlepiej w zbiorach 1 i 3. Algorytm Chan'a okazał się wyjątkowo skuteczny w zbiorze 4, gdzie wszystkie punkty należały do otoczki wypukłej. Z kolei algorytmy Jarvisa i przyrostowy wykazały wyraźne ograniczenia w przypadkach, gdzie ich złożoność teoretyczna osiąga swoje maksimum. Zadziwiające mogą być wyniki osiągnięte przez algorytm Chan'a. Mimo, iż jego złożoność jest najlepsza, to poprzez nieoptymalną aproksymację oraz możliwą implementację niespodziewanie przegrywa z innymi rozwiązaniami. Pozostałe algorytmy wykazywały się dobrymi rezultatami czasowymi, jednakże nie stanowiły one najbardziej optymalnych rozwiązań.

Bibliografia, inspiracje programistyczne



- Wikipedia, Quickhull, dostępne pod adresem: <https://pl.wikipedia.org/wiki/Quickhull> (Dostęp: 3 stycznia 2025)
- Misha Kazhdan, QuickHull Algorithm Lecture Notes, dostępne pod adresem: <https://www.cs.jhu.edu/~misha/Spring16/07.pdf> (Dostęp: 3 stycznia 2025)
- GeeksforGeeks, Convex Hull using Divide and Conquer Algorithm, dostępne pod adresem: <https://www.geeksforgeeks.org/convex-hull-using-divide-and-conquer-algorithm/> (Dostęp: 3 stycznia 2025)
- Wikipedia, Chan's Algorithm, dostępne pod adresem: https://en.wikipedia.org/wiki/Chan%27s_algorithm (Dostęp: 3 stycznia 2025)
- Pranay Yadav, Repozytorium Convex-Hull, dostępne pod adresem: <https://github.com/ypranay/Convex-Hull/blob/master/ChansAlgorithmForConvexHull.cpp> (Dostęp: 4 stycznia 2025)
- Tom Switzer, implementacja algorytmu Chan'a, dostępne pod adresem: <https://gist.github.com/tixxit/252229> (Dostęp: 4 stycznia 2025)
- Koło Naukowe Bit, przykłady testowe, dostępne pod adresem: <https://github.com/aghbit/Algorytmy-Geometryczne/blob/master/bitalg/lab2/main.ipynb> (Dostęp: 4 stycznia 2025)